

Informe Técnico

Inducción de reglas a partir de ejemplos positivos y negativos
Caso: compra de automóvil eléctrico

Técnica: inducción de reglas por diferencia de valores
(atributos y valores exclusivos de la clase positiva)

1. Introducción

Este informe documenta y analiza un script en Python que implementa un algoritmo simple de **inducción de reglas**, una técnica de aprendizaje simbólico utilizada para inferir condiciones lógicas que permiten distinguir ejemplos positivos de ejemplos negativos dentro de un conjunto de datos.

El caso analizado busca identificar qué características determinan que una persona pueda comprar un automóvil eléctrico, comparando un conjunto de ejemplos positivos (personas que sí compraron) con un conjunto de ejemplos negativos (personas que no compraron).

2. Conjunto de datos

El script define dos listas de diccionarios: *positivos* (3 ejemplos) y *negativos* (3 ejemplos), cada uno con los atributos *edad*, *ingreso*, *tiene_garaje* y *distancia_trabajo*.

2.1 Ejemplos positivos (compraron el auto eléctrico)

#	Edad	Ingreso	Tiene garaje	Distancia al trabajo
1	Joven	Alto	Si	Corta
2	Adulto	Medio	Si	Media
3	Mayor	Alto	Si	Larga

2.2 Ejemplos negativos (no compraron el auto eléctrico)

#	Edad	Ingreso	Tiene garaje	Distancia al trabajo
1	Joven	Alto	No	Corta
2	Adulto	Medio	No	Media
3	Mayor	Bajo	No	Larga

3. Código fuente analizado

El algoritmo funciona en los siguientes pasos: (1) toma los nombres de atributos a partir del primer ejemplo positivo; (2) para cada atributo, recolecta los valores únicos presentes en positivos y en negativos; (3) conserva únicamente los valores que aparecen en positivos pero **nunca** en negativos; (4) si para un atributo existe al menos un valor exclusivo de la clase positiva, lo incorpora a la regla inducida.

```
# Obtener los nombres de los atributos
atributos = []
ejemplo = positivos[0]
for clave in ejemplo:
    atributos.append(clave)

# Para cada atributo, comparar valores únicos en positivos y negativos
for atributo in atributos:
    valores_pos = []
    valores_neg = []

    for ej in positivos:
        valor = ej[atributo]
        if valor not in valores_pos:
            valores_pos.append(valor)

    for ej in negativos:
        valor = ej[atributo]
        if valor not in valores_neg:
            valores_neg.append(valor)

    # Conserva los valores que están en positivo pero no en negativo
    valores_validos = []
    for valor in valores_pos:
        encontrado = False
        for v in valores_neg:
            if valor == v:
                encontrado = True
                break
        if not encontrado:
            valores_validos.append(valor)

    if len(valores_validos) > 0:
        regla[atributo] = valores_validos
```

4. Fundamento del método

El enfoque implementado es una forma elemental de **inducción por diferencia** (similar en espíritu al algoritmo *Find-S* o a la búsqueda de literales discriminantes en FOIL). Para cada atributo, se identifican los valores que sólo aparecen entre los ejemplos positivos: si un valor nunca se observa en la clase negativa, se considera un buen indicador (candidato de regla) para predecir la clase positiva.

Formalmente, para un atributo A con valores observados $V_{\text{pos}}(A)$ en los ejemplos positivos y $V_{\text{neg}}(A)$ en los negativos, el algoritmo calcula:

$$\text{Valores_válidos}(A) = V_{\text{pos}}(A) - V_{\text{neg}}(A)$$

Si este conjunto no está vacío, el atributo A se incorpora a la regla inducida junto con sus valores exclusivos.

5. Resultados de la ejecución

Se ejecutó el script sobre el conjunto de datos descrito. El análisis atributo por atributo arrojó lo siguiente:

Atributo	Valores en positivos	Valores en negativos	Valores exclusivos de positivos
edad	Joven, Adulto, Mayor	Joven, Adulto, Mayor	— (ninguno)
ingreso	Alto, Medio	Alto, Medio, Bajo	— (ninguno)
tiene_garaje	Si	No	Si
distancia_trabajo	Corta, Media, Larga	Corta, Media, Larga	— (ninguno)

Regla inducida por el script:

tiene_garaje debe ser igual a: ['Si']

Es decir, el algoritmo determina que el único atributo con un valor exclusivo de la clase positiva es **tiene_garaje**, con el valor **"Si"**.

6. Interpretación de los resultados

Los atributos *edad* y *distancia_trabajo* presentan exactamente los mismos valores en ambas clases (Joven/Adulto/Mayor y Corta/Media/Larga aparecen tanto en compradores como en no compradores), por lo que no aportan ningún valor discriminante y quedan excluidos de la regla.

El atributo *ingreso* tampoco resulta discriminante: si bien la clase positiva sólo presenta los valores "Alto" y "Medio", ambos también aparecen en la clase negativa (que además incluye "Bajo"), por lo que ningún valor de ingreso es exclusivo de los compradores.

El atributo *tiene_garaje* es el único verdaderamente discriminante: los tres ejemplos positivos tienen garaje ("Si"), mientras que los tres ejemplos negativos no lo tienen ("No"). Esto separa perfectamente ambas clases en el conjunto de datos, por lo que el script induce la regla **tiene_garaje = "Si"** como condición suficiente, según estos datos, para predecir la compra del automóvil eléctrico.

Es importante señalar que, al tratarse de un conjunto de datos muy pequeño (3 positivos y 3 negativos), la regla obtenida podría no generalizar bien a nuevos casos; el atributo *tiene_garaje* podría estar actuando como variable de confusión (por ejemplo, correlacionada con el poder adquisitivo real) más que como causa directa de la compra.

7. Conclusión

El script implementa correctamente un algoritmo de inducción de reglas por diferencia de valores entre clases. Aplicado al conjunto de datos sobre compra de automóviles eléctricos, el único atributo que separa perfectamente a los ejemplos positivos de los negativos es **tiene_garaje**, con el valor **"Si"**. Este resultado ilustra cómo técnicas simples de inducción pueden extraer reglas interpretables a partir de ejemplos etiquetados, aunque su validez depende fuertemente del tamaño y la representatividad del conjunto de datos utilizado.